**FPT TELECOM**




# Direct Memory Access Controller

## HARDWARE DESIGN SPECIFICATION (HDS)

---

V1.0

March 20, 2026

# Contents

Contents 2

Reversion and History 4

Table of Tables 5

Table of Figures 6

1. Overview 7

1.1. Introduction 7

1.2. Compliances 8

2. Supported Features 8

3. Unsupported Features 8

4. Block Diagram 8

5. Parameters 9

6. Functions 10

6.1. Clock 10

6.2. Reset 10

6.3. Function 1 10

6.4. Function 2 10

6.5. Function 3 10

7. Interfaces 10

7.1. APB Interface 10

7.2. UART Interface 10

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

**8. REGISTERS 10**

**8.1. Register Symbol 10**

**8.2. Register Table 10**

**9. OPERATION 10**

**9.1. Initial Sequence 10**

**9.2. Interrupt Sequence 10**

**9.3. DMA Sequence 10**

**10. VERIFICATION NOTES 11**

**11. INTEGRATION 11**

**11.1. Primitive Modules 11**

**11.2. SDC Constraint 11**

**APPENDIX A. ACRONYMS 14**

**APPENDIX B. REVIEW 15**

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

# Reversion and History

| Version | Date (mm/dd/yyyy) | Author | Reviewer | Description of Change |
|---|---|---|---|---|
| V1.0 | Mar 27, 2026 | Danhht3 | - | First version |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

# Table of Tables

Table 5-1. Parameter Table 10

Table 11-1. Design Review 1 15

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

# Table of Figures

Figure 4-1. Block Diagram 9

Figure 11-1. Max Delay Constraint of JTAG-to-AXI Bridge 12

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

# 1. Overview

## 1.1. Introduction

This document describes the ftel_dma IP, a configurable, register-programmed Direct Memory Access (DMA) controller designed to offload data movement operations from the host processor within an SoC. The ftel_dma IP autonomously transfers data between memory and peripherals, or between memory locations, without requiring continuous processor intervention for each data element.

In modern SoC designs, the processor is frequently burdened with repetitive data movement tasks such as transferring data between memory and peripheral FIFOs, or relocating blocks of data within memory. Performing these transfers directly via the processor consumes significant CPU cycles, increases interrupt overhead, and limits overall system throughput. The ftel_dma IP addresses this problem by taking ownership of bulk data movement, thereby reducing CPU overhead, improving system throughput, and enabling the processor to perform other tasks concurrently with ongoing data transfers.

The ftel_dma IP is required in the SoC to provide efficient, low-latency, and low-CPU-overhead data movement between system memory and peripheral devices, as well as between memory regions. Its primary purpose is to serve as a dedicated data-movement engine that is configured by software but operates independently once a transfer is initiated.

Within the system, ftel_dma occupies the role of a shared, memory-mapped peripheral. Software configures and controls the DMA through an AMBA APB register interface, programming transfer parameters such as source address, destination address, transfer size, and transfer length. Once configured and enabled, the DMA Engine performs the actual data movement autonomously through an AMBA AXI 4 master interface, generating read and write transactions to memory and peripherals on behalf of the processor. This separation of control path (APB) and data path (AXI 4) allows software to manage multiple channels efficiently while the DMA hardware handles the high-bandwidth transfer activity.

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

The ftel_dma IP is intended for use in a broad range of embedded and ASIC applications, including microcontrollers, IoT devices, industrial control systems, communication subsystems, and consumer electronics, wherever efficient offloading of data movement between memory and peripherals is required.

### 1.2. Compliances

The ftel_dma IP complies with the following standards and specifications:

- AMBA® APB Protocol Specification, for the register-programming interface used by software to configure and control the DMA
- AMBA® AXI 4 Protocol Specification, for the AXI 4 master interface used to perform data-transfer transactions with system memory and peripherals
- The project-specific register-programmed DMA programming model, as defined by the ftel_dma register set and channel control mechanism described in this document

## 2. Supported Features

The ftel_dma is a configurable, register-programmed Direct Memory Access Controller intended for data movement within an MCU or embedded SoC. The IP separates the software-programming path from the high-bandwidth data-transfer path. Software configures each DMA channel through the APB slave interface, while the DMA hardware performs memory and peripheral accesses through the shared AXI master interface.

Unless otherwise stated, each feature described in this section applies independently to every implemented DMA channel. Integration-time capabilities such as the number of channels, AXI data width, FIFO depth, and maximum outstanding depth are reported to software through the DMA capability registers.

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

**Table 2-1 Summarizes the supported features.**

| No | Feature | Supported Capability |
|---|---|---|
| 1 | DMA architecture | One active DMA core containing both AXI read and AXI write engines |
| 2 | DMA channels | Eight implemented DMA channels |
| 3 | Channel FIFO | One dedicated 32-byte data FIFO for each DMA channel |
| 4 | Transfer directions | Memory-to-Memory, Memory-to-Peripheral, and Peripheral-to-Memory |
| 5 | APB3 programming interface | 32-bit APB3 interface for configuration, control, status, and interrupt access |
| 6 | AXI4 master interface | One 32-bit AXI4-compatible memory-mapped master interface with independent read and write channels |
| 7 | Source and destination addresses | Independently programmable source and destination addresses for every channel |
| 8 | Addressing modes | Independent incrementing or fixed-address operation for source and destination |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | |
|---|---|---|
| 9 | Transfer data-unit size | 8-bit, 16-bit, and 32-bit transfer units |
| 10 | Transfer size | Programmable transfer amount through a 10-bit channel buffer-size field |
| 11 | AXI burst transfer | Programmable read and write burst generation with a maximum AXI burst length of 16 beats |
| 12 | Software start | Independent software start command for every DMA channel, with an additional global channel-start register |
| 13 | Peripheral request selection | Independent receive-side and transmit-side peripheral request selection for each channel |
| 14 | External peripheral requests | Up to 31 receive-side and 31 transmit-side peripheral request inputs |
| 15 | Software-generated peripheral request | Software request bits are OR-combined with the corresponding external peripheral request signals |
| 16 | Peripheral request clear | A clear pulse is generated for the selected peripheral request after the associated DMA operation completes |
| 17 | Peripheral request timing control | Programmable request delay and request-block control are provided independently for the read and write sides |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | |
|---|---|---|
| 18 | Channel arbitration | Independent read-side and write-side arbitration among eligible channels |
| 19 | Channel priority control | Programmable priority selection for the shared AXI read and write arbitration paths |
| 20 | Read/write overlap | AXI read and write operations may overlap because the two paths are decoupled through the channel FIFO |
| 21 | Outstanding support | The AXI read and write engines each support up to four accepted transactions awaiting completion. The effective per-channel limit is programmable from one to four and is additionally constrained by global context depth, FIFO capacity, endpoint type, and arbitration. |
| 22 | Narrow and partial transfers | Address alignment, byte-lane selection, data slicing, and AXI write strobes support sub-word transfers |
| 23 | AXI error detection | Detection of AXI read and write SLVERR and DECERR responses |
| 24 | FIFO error detection | Detection of channel FIFO overflow and underflow conditions |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | |
|---|---|---|
| 25 | AXI timeout monitoring | Timeout monitoring for selected AXI address, data, and response operations |
| 26 | Channel watchdog | Watchdog monitoring for channels that remain active without expected progress |
| 27 | Interrupt generation | Per-channel raw status, interrupt enable, interrupt clear, masked status, and consolidated interrupt output |
| 28 | Runtime status | Channel active state, command count, FIFO fullness, interrupt state, and selected error conditions |
| 29 | Capability reporting | Read-only user-definition registers report the fixed RTL configuration |

### 2.1 8-DMA Channels

The current RTL implements eight DMA channels.

Each channel contains an independent:

- APB-visible configuration context.
- Source and destination transfer configuration.
- Transfer-progress state.
- Data FIFO.
- Peripheral-request selection.
- Interrupt raw status and interrupt enable state.
- Command and interrupt counters.

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

All eight channel modules are instantiated directly inside. The global start register provides an eight-bit start bitmap, allowing software to request one or more channels independently.

The number of channels is fixed in the current RTL build. It is not selected through a Verilog parameter.

No channel is permanently tied to a particular peripheral. Software selects the source-side and destination-side peripheral numbers through the channel static-configuration registers.

### 2.2 Dedicated FIFO per Channel

Each DMA channel contains one dedicated data FIFO between its read path and write path.

The conceptual datapath is:

Source AXI READ -> Channel Data FIFO -> Destination AXI Write

The current FIFO uses a five-bit byte pointer and stores eight 32-bit words:

FIFO capacity = 8 words x 4 bytes = 32 bytes per channel

The FIFO provides:

- Independent read and write pointers.
- Byte-level write selection.
- FIFO fullness calculation.
- Space-available indication.
- Read-data-valid indication.
- Overflow detection.
- Underflow detection.

The FIFO decouples source-side and destination-side latency. For example, an M2P channel may read data from memory before the destination peripheral becomes ready, provided that sufficient FIFO capacity remains.

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

## 2.3 Supported Transfer Directions

The current supported transfer directions are:

1. Memory-to-Memory.
2. Memory-to-Peripheral.
3. Peripheral-to-Memory.

Peripheral number zero represents memory and is treated as permanently ready. Peripheral numbers 1 through 31 select external peripheral request inputs.

### 2.3.1 Memory-to-Memory

For a Memory-to-Memory transfer:

Read peripheral number = 0

Write peripheral number = 0

No external peripheral request is required.

The normal address modes are:

Source = Increment

Destination = Increment

The DMA reads data from the programmed source region, stores it in the channel FIFO, and writes it to the destination region.

### 2.3.2 Memory-to-Peripheral

For a Memory-to-Peripheral transfer:

Read peripheral number = 0

Write peripheral number = 1–31

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

The normal address modes are:

Source = Increment

Destination = Fixed

The DMA may prefetch data from memory into the channel FIFO. Destination writes are allowed only while the selected transmit request is active.

After the AXI write response has been accepted, the DMA generates a one-clock transmit-clear pulse for the selected peripheral.

For M2P operation, the memory-read side may maintain up to four accepted AXI read transactions, subject to the configured read pending limit and the 32-byte channel FIFO reservation. The peripheral-write side remains single-credit because the current peripheral adapter stores one request at a time.

### 2.3.3 Peripheral-to-Memory

For a Peripheral-to-Memory transfer:

Read peripheral number = 1–31

Write peripheral number = 0

The normal address modes are:

Source = Fixed

Destination = Increment

The DMA waits for the selected receive request before issuing the peripheral-side AXI read.

After the final read-data beat of the issued command has been accepted, the DMA generates a one-clock receive-clear pulse for the selected peripheral.

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

For P2M operation, the peripheral-read side accepts one request credit at a time. After received data is buffered, the memory-write side may maintain up to four accepted AXI write transactions awaiting B responses.

### 2.4 APB Programming Interface

The DMA provides a 32-bit APB slave interface for software configuration and status access.

The APB interface provides access to:

- Global configuration registers.
- User-definition and capability registers.
- Global channel-start control.
- Software-generated peripheral requests.
- Per-channel command registers.
- Per-channel static-configuration registers.
- Channel enable and start control.
- Channel active and FIFO status.
- Interrupt raw status.
- Interrupt clear.
- Interrupt enable.
- Masked interrupt status.

The APB interface carries only configuration and status information. DMA payload data is transferred through the AXI master interface.

### 2.5 AXI Master Interface

The current implementation provides one 32-bit AXI memory-mapped master interface.

The interface contains:

- Write-address channel.
- Write-data channel.
- Write-response channel.

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

- Read-address channel.
- Read-data channel.

The implemented widths are:

| Property | Current RTL |
|---|---|
| Address width | 32 bits |
| Data width | 32 bits |
| Write strobe width | 4 bits |
| AXI ID width | 4 bits externally |
| Burst-length field | 4 bits |
| Maximum encoded burst | 16 beats |

The presence of **WID** and a four-bit burst-length field indicates an AXI3-style interface rather than a native AXI4 interface.

Read and write paths can operate concurrently because they are implemented as separate engines and are decoupled through the channel FIFOs.

| Property | Current RTL |
|---|---|
| External AXI ID | Fixed 4'h4 |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | |
|---|---|
| Global read pending depth | 4 |
| Global write pending depth | 4 |
| Read response routing | In-order context FIFO |
| Write response routing | In-order context FIFO |
| Write-data interleaving | Not supported |
| LOCK | Normal access only |
| BURST | FIXED, INCR |
| WRAP | Unsupported |

All transactions use the same external ID. Responses are associated with DMA channels by internal in-order context FIFOs. RID and BID are checked against 4'h4 but are not used to select a channel.

### 2.6 Transfer Data-Unit Size

The channel datapath supports:

- 8-bit units.
- 16-bit units.
- 32-bit units.

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

The selected transfer size controls:

- AXI ARSIZE and AWSIZE.
- Address increment amount.
- Read-data slicing.
- Write-data alignment.
- AXI WSTRB generation.
- FIFO pointer advancement.

For a 32-bit AXI bus:

| Data-unit size | Bytes per unit | Typical use |
|---|---|---|
| 8 bits | 1 | UART byte |
| 16 bits | 2 | ADC or audio sample |
| 32 bits | 4 | Memory word or SPI word |

### 2.7 Programmable Transfer Amount

The programmed transfer amount is stored in the channel command buffer-size field.

The current field is ten bits wide. Therefore, the HDS should not state a fixed limit of 256 units unless the software-visible encoding is deliberately restricted to eight bits.

A safe specification statement is:

> Each channel supports a programmable transfer amount through the 10-bit command buffer-size field. The exact interpretation and legal range shall be defined by the register-programming model.

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

This statement matches the RTL more accurately than:

1 to 256 data units per transfer.

The implementation also contains a remaining-transfer block that tracks source-side and destination-side progress independently.

### 2.8 Channel Enable and Start

Each channel supports:

- Channel enable.
- Per-channel start register.
- Global start bitmap.
- Active-state tracking.
- Independent read-active and write-active tracking.
- Transfer-end tracking.
- Command and completion counters.

Software normally performs:

Program command registers

->

Program static configuration

->

Enable channel

->

Write channel start

->

Monitor active/status/interrupt

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

A dedicated, explicitly named abort command is not evident in the current register interface. Therefore, abort and graceful-disable behavior should not be listed as supported until their precise control mechanism has been defined and verified.

### 2.9 Channel Arbitration

The eight channels share the central AXI read and write engines.

Two independent arbiters are used:

- Read-channel arbiter.
- Write-channel arbiter.

A channel becomes eligible when it is active and its read-side or write-side readiness conditions are satisfied.

Read and write arbitration may select different channels during the same period. For example, the read engine may be filling Channel 1 FIFO while the write engine drains Channel 3 FIFO.

Priority-related ports exist in the RTL, but the current configuration is fixed to support only weighted arbitration. Therefore, the supported behavior is limited to weighted round-robin without software-programmable fixed priority.

### 2.10 Peripheral Handshake

The peripheral adapter accepts one request while in REQ_IDLE. It latches the request type, asserts the internal request until the DMA service completes, then holds the acknowledgement valid until the peripheral accepts it.

The current adapter supports one outstanding peripheral request credit. DTYPE_FLUSH is acknowledged without generating an AXI data transfer. DTYPE_SINGLE and DTYPE_BURST are preserved as request metadata; the current implementation does not queue multiple burst credits.

The DRLAST input is retained as interface metadata but is not used to control transfer termination in the current RTL.

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| Slot | DMA0 RX | DMA 0 TX | DMA1 RX | DMA 1 TX |
|---|---|---|---|---|
| 1 | SPI 0 RX | SPI0 TX | USART 2RX | USART 2 TX |
| 2 | USART 0 RX | USART 0 TX | USART 3 RX | USART 3 TX |
| 3 | USART 1 RX | USART 1 TX | USART 4 RX | USART 4 TX |
| 4 | QSPI RX | QSPI TX | I2C RX | I2C TX |
| 5 | - | - | SPI 0 RX | SPI0 TX |

### 2.11 Independent Read and Write Operation

The AXI read and write engines operate independently.

A channel may read source data while previously read data is being written to the destination, provided that:

- Channel FIFO space is available.
- Source request conditions are satisfied.
- The read engine accepts the channel.
- No blocking error condition is active.

Similarly, the write engine may drain one channel FIFO while the read engine services another channel.

Read and write operation may overlap, and each AXI engine supports up to four accepted transactions awaiting completion. The effective depth is controlled independently for the read and write endpoints. Peripheral endpoints remain single-credit in the current adapter implementation.

### 2.12 Narrow Transfer, Byte Strobes

The RTL contains:

- Read-data slicer.
- Write-data slicer.

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

- Address-offset calculation.
- AXI write-strobe generation.
- Byte-selected FIFO writes.
- A programmable two-bit endian-swap field.
- A 32-bit swap primitive.

These blocks support narrow 8-bit and 16-bit transfers on the 32-bit AXI datapath and partial byte-lane writes.

The presence of this logic does not by itself prove correct behavior for every arbitrary unaligned source/destination combination. Unaligned corner cases should remain in the verification-required category.

For narrow INCR write bursts, WSTRB is generated for each accepted beat from the initial destination lane, transfer size, burst type, and current write-beat index.

For FIXED bursts, the byte lane and WSTRB remain constant throughout the burst.

| **Transfer size** | **Valid WSTRB** |
|---|---|
| 8-bit | 0001, 0010, 0100, 1000 according to byte lane |
| 16-bit | 0011 or 1100 |
| 32-bit | 1111 |

Odd-address 16-bit writes and non-word-aligned 32-bit writes generate an internal write-protocol error. Arbitrary unaligned accesses that cross a 32-bit word boundary are not claimed as supported until verified.

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

### 2.13 Timeout and Watchdog Monitoring

The RTL contains AXI timeout and channel-watchdog modules.

The AXI timeout logic monitors selected AXI valid/ready wait conditions, including read-address, write-address, and write-data phases. Additional channel response tracking contributes timeout and error interrupt sources.

The watchdog monitors active channels and reports a channel that fails to make expected progress.

A dedicated timer specifically measuring how long a channel waits for a peripheral request was not identified as an independent peripheral-request timeout block. Therefore, the supported statement should be:

> AXI transaction timeout and channel watchdog monitoring are supported. Dedicated peripheral-request timeout behavior requires further confirmation.

### 2.14 Error Detection

The current RTL detects the following error conditions:

- AXI RID mismatch
- AXI BID mismatch
- Early or missing RLAST
- Write-data fetch-credit overflow/underflow
- Write-data FIFO overflow/underflow
- Multiple channel FIFO read requests or returns in one cycle
- Invalid write-strobe alignment

These errors are routed to per-channel status and interrupt logic.

The RTL does not provide sufficiently clear evidence for all of the following HDS claims:

- Full illegal-configuration validation.
- Start-time source and destination alignment rejection.

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

- Invalid peripheral ID rejection.
- Multiple-channel ownership conflict detection.
- Peripheral release timeout.
- Outstanding owner mismatch detection while outstanding mode is disabled.

Those items should not be listed as fully supported until dedicated validation logic and tests exist.

### 2.15 Interrupt Generation

Each channel provides:

- Raw interrupt status.
- Interrupt clear.
- Interrupt enable.
- Masked interrupt status.

The current channel interrupt bus includes multiple event and error sources, including:

- Command completion when enabled by the command.
- AXI read/write error.
- AXI timeout.
- Watchdog timeout.
- FIFO overflow.
- FIFO underflow.
- Additional generated transfer-status conditions.

Per-channel masked interrupts are combined into a consolidated DMA interrupt output.

The current top-level provides one interrupt output group rather than a separate top-level pin for every channel.

### 2.16 Capability Reporting

Read-only user-definition registers report the generated RTL configuration, including:

- Eight channels.

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

- 32-bit AXI bus.
- 32-bit address width.
- FIFO size encoding.
- Command FIFO depth.
- Buffer-size field width.
- Peripheral-handshake support.
- Timeout support.
- Watchdog support.
- Command-list support.
- Simultaneous read/write support.
- Outstanding support enabled.
- Maximum read pending depth = 4.
- Maximum write pending depth = 4.
- Independent read/write operation enabled.
- Peripheral handshake enabled.
- Priority control disabled unless the current RTL build explicitly
- implements and connects the priority fields.
- Block mode disabled.
- Clock-divider support disabled.

## 3. Unsupported Features

- The current ftel_dma RTL does not support:
- Scatter-gather descriptor walking beyond the current command-chain mechanism
- Circular buffer mode
- Double-buffer mode
- 2D or stride transfer mode
- AXI WRAP bursts
- AXI exclusive accesses
- AXI locked sequences
- Write-data interleaving between AXI IDs

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

- Arbitrary unaligned transfers crossing a 32-bit word boundary
- Software-programmable secure/non-secure memory-region enforcement

# 4. Block Diagram

## 4.1 Block Description

The **FTEL DMAC** consists of the functional blocks shown in Figure 4-1. The controller is programmed through the **APB bus**, receives optional DMA requests from external **peripherals**, and performs data movement through the **AXI bus** using the internal DMA Engine.

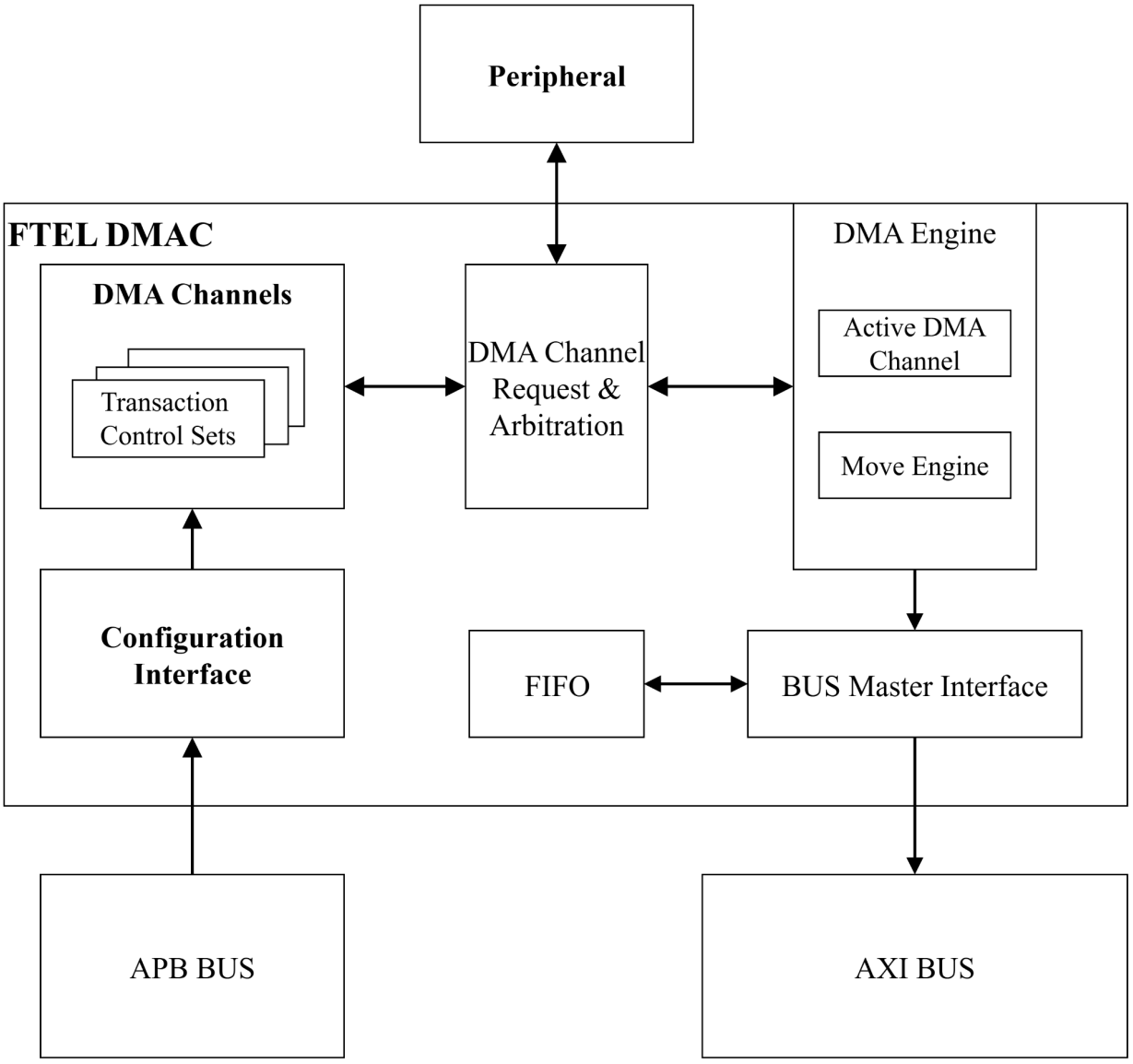



**Figure 4-1:** DMAC Block Design

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

- **Configuration Interface** – The Configuration Interface implements the APB slave programming path of the DMAC. It decodes APB read/write accesses and updates the internal control and status registers. This block provides access to global control fields and per-channel configuration fields.
- **DMA Channels** – The DMA Channels block contains the programmable contexts of all DMA channels. The number of channels is parameter configurable from **1 to 8**. Each channel can be individually enabled, requested, completed, aborted, or reported as error.
- **Transaction Control Sets** – Each DMA channel has one Transaction Control Set that stores the transfer context required by the DMA Engine. The TCS includes source address, destination address, transfer direction, transfer size, transfer length, address increment control, outstanding setting, interrupt enable, and timeout configuration. The supported transfer directions are **memory-to-memory**, **memory-to-peripheral**, and **peripheral-to-memory**. Transfer size is programmable as **8-bit, 16-bit, or 32-b**it, and transfer length is programmable up to 256 transfer units.
- **Peripheral** – The Peripheral block represents external IPs that can request DMA service. For peripheral-triggered transfers, the peripheral sends a DMA request to the DMAC, and the corresponding channel is serviced when it is enabled and selected by the arbitration logic.

- **DMA Channel Request and Arbitration** – This block collects pending requests from all enabled DMA channels. Requests may come from software through the Configuration Interface or from external peripherals. When more than one channel is pending, the arbitration logic selects one channel according to the implemented priority or arbitration scheme and forwards it as the active channel to the DMA Engine.
- **DMA Engine** – The DMA Engine executes the actual data-transfer operation for the selected channel. It loads the active channel context, controls the transfer sequence, generates read/write operations, updates source and destination addresses, tracks the remaining transfer length, monitors outstanding transactions, controls FIFO operation, and detects timeout or error conditions.

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

    - **Active DMA Channel** – The Active DMA Channel block stores the currently selected channel ID and its active transfer context. The active context remains stable while the transfer is running and is released only when the transfer is completed, aborted, timed out, or terminated by an error.
    - **Move Engine** – The Move Engine performs the data movement operation. It issues read commands from the source side, writes data to the destination side, updates the transfer counter, applies address increment rules, and checks transfer completion. It uses the FIFO as an intermediate buffer between the read path and write path.
- **FIFO** – The FIFO block provides temporary data buffering between the AXI read and write paths. In this design, each DMA channel has one dedicated data FIFO. The FIFO decouples read latency from write latency and provides full, empty, overflow, and underflow status to the DMA Engine.
- **Bus Master Interface** – The Bus Master Interface converts internal DMA read/write commands into AXI master transactions. It issues AXI read and write accesses to system memory or memory-mapped peripherals according to the active channel configuration, including address, transfer size, transfer length, and outstanding control.
- **Interrupt Interface** – Aggregates per-channel interrupt sources (completion, error, timeout) and drives the consolidated interrupt output, toward the system interrupt controller.

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

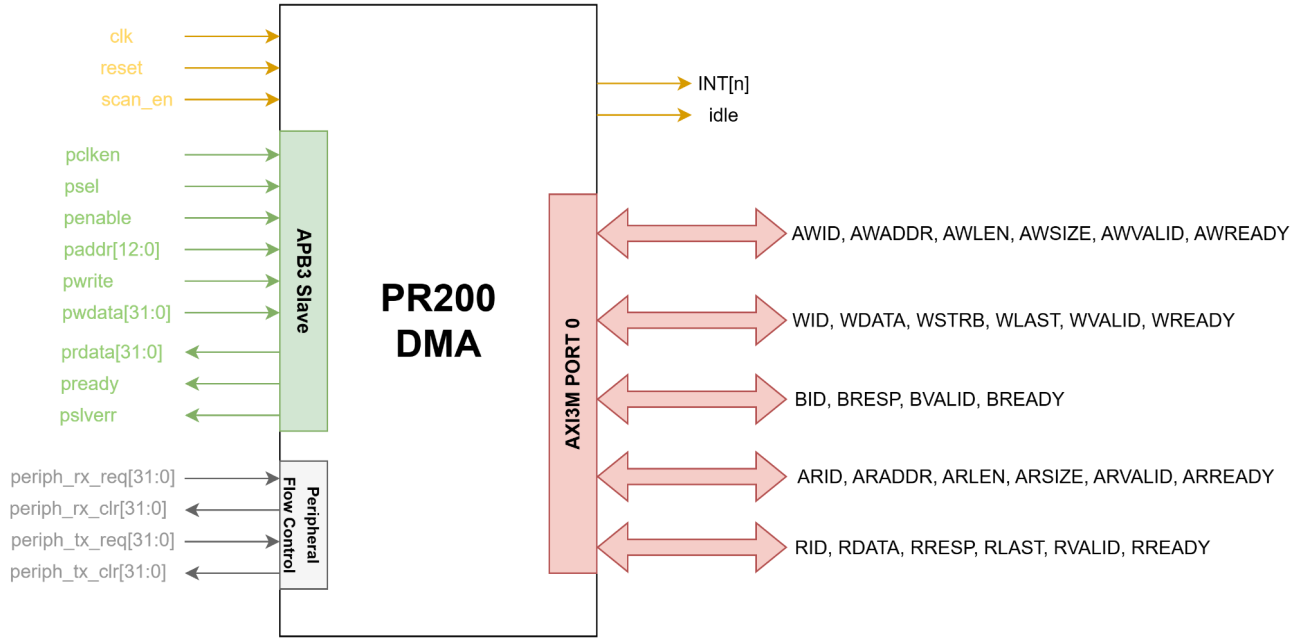
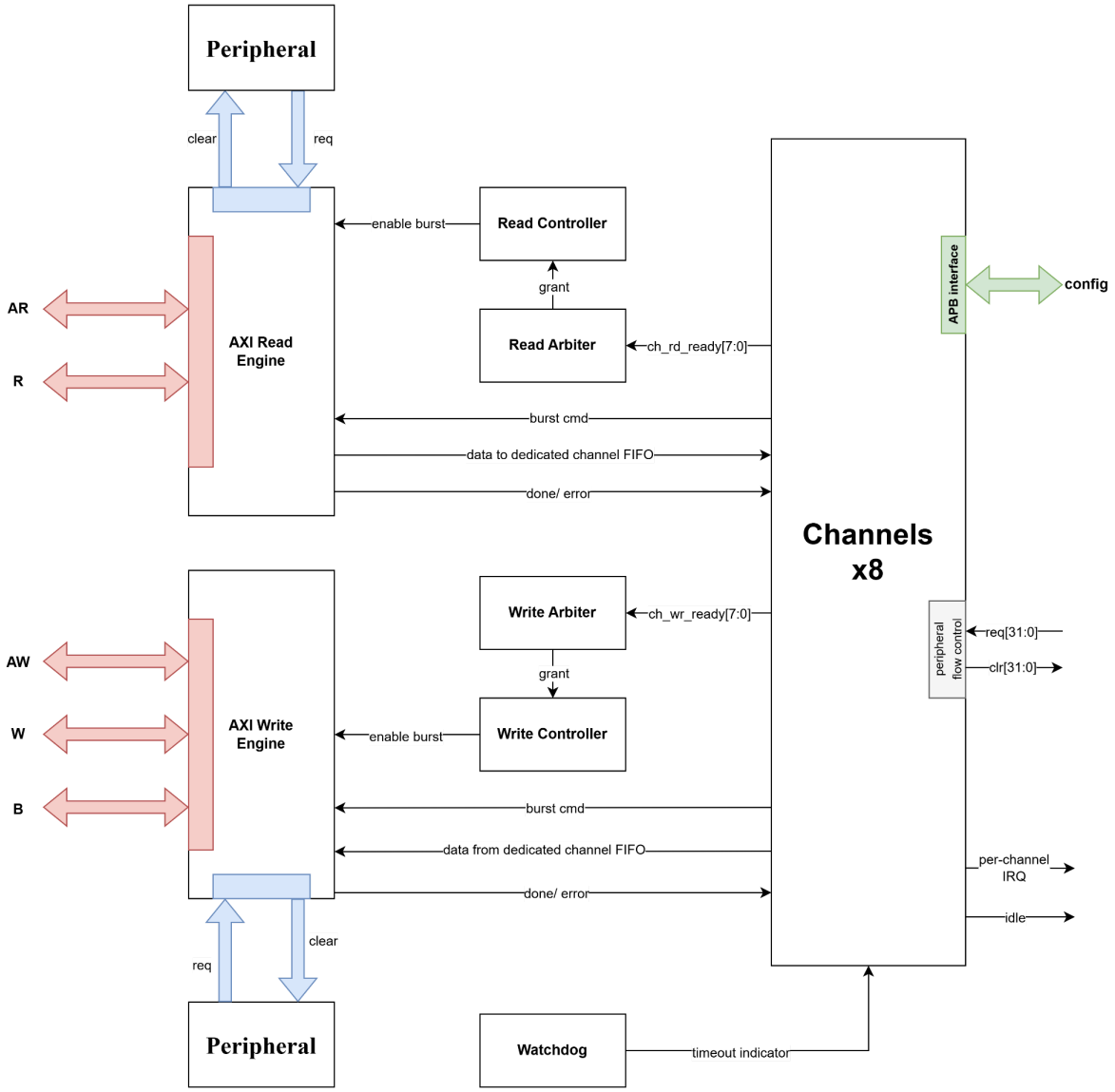



**Figure 4-1:** DMAC Pinout

Copyright © 2026 FPT Telecom. All rights reserved. Confidential




# 5. Parameters

**Table 5-1. Parameter Table**

| No. | Parameter | Value | Description |
|---|---|---|---|
| 1 | PARA_AXI_DATA_WIDTH | 32 | AXI read/write data width. Current RTL supports 32-bit only. |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | | |
|---|---|---|---|
| 2 | PARA_AXI_ADDR_WIDTH | 32 | AXI address width. |
| 3 | PARA_AXI_ID_WIDTH | 4 | External AXI ID width. |
| 4 | PARA_AXI_MASTER_ID | 4 | Fixed ID driven on ARID, AWID and WID. |
| 5 | PARA_CHANNEL_COUNT | 8 | Fixed channel count in this build. |
| 6 | PARA_CHANNEL_FIFO_BYTES | 32 | Data FIFO capacity per channel. |
| 7 | PARA_AXI_MAX_RD_PENDING | 4 | Global accepted read transactions awaiting completion. |
| 8 | PARA_AXI_MAX_WR_PENDING | 4 | Global accepted write transactions awaiting B response. |
| 9 | PARA_AXI_MAX_BURST_BEATS | 16 | AXI3 maximum burst length. |
| 10 | PARA_AXI_MEM_ARCACHE | Integration-defined | Default memory read cache attribute. |
| 11 | PARA_AXI_MEM_AWCACHE | Integration-defined | Default memory write cache attribute. |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | | |
|---|---|---|---|
| 12 | PARA_AXI_PERIPH_ARCACHE | Integration-defined | Peripheral read attribute. |
| 13 | PARA_AXI_PERIPH_AWCACHE | Integration-defined | Peripheral write attribute. |
| 14 | PARA_AXI_MEM_ARPROT | Integration-defined | Memory read protection attribute. |
| 15 | PARA_AXI_MEM_AWPROT | Integration-defined | Memory write protection attribute. |

### 5.1. Clock

The ftel_dma operates in one synchronous clock domain. The APB register interface, DMA channel logic, peripheral adapters, and AXI master logic use the same DMA clock. Legacy clock-enable inputs are tied active in the current integration.

### 5.2. Reset

Active-low synchronous/asynchronous reset according to actual RTL port. Reset clears channel activity, outstanding counters, AXI context FIFO, FIFO reservation state, write-data fetch credits, peripheral ownership, adapter request state, interrupt raw status, and valid outputs.

### 5.3. Channel Scheduling and Arbitration

### 5.4. AXI Read/Write Transaction Management

### 5.5. Peripheral Request Service

### 5.6. FIFO Reservation and Data Buffering

### 5.7. Error, Timeout and Interrupt Handling

## 6. Interfaces

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

### 6.1. APB Interface

### 6.2. AXI Interface

### 6.3. Peripheral Interface

## 7. Registers

The ftel_dma register set consists of:

- One global register bank.
- Eight identical channel register banks.
- One 256-byte address region for each channel.
- One 256-byte global register region.

For Channel n, where n=0...7:

Channel Register Address =

DMA_BASE + (0x100 × n) + Channel_Register_Offset

Global register address:

Global Register Address =

DMA_BASE + 0x1000 + Global_Register_Offset

**Table 7-1. DMA Global Register Map**

| Offset | Register | Type | Width | Reset | Description |
|---|---|---|---|---|---|
| 0x1000 | CORE_STATUS | RO | 32 | 0X0 | Consolidated channel interrupt status. Bits [7:0] correspond to Channels 7 through 0. Bit **n** is set when Channel n has at least one enabled interrupt source. Bits [31:8] read as zero. |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | | | | |
|---|---|---|---|---|---|
| 0x1004–0x102C | Reserved | — | — | — | Unmapped address range. APB access returns a slave error. |
| 0x1030 | CORE_JOINT_CONFIG | RW | 32 | 0X0 | Compatibility register retained from the original register map. Joint mode is not implemented in the current RTL. Reads return zero and writes have no functional effect. |
| 0x1034 | Reserved | — | — | — | Unmapped. APB access returns a slave error. |
| 0x1038 | CORE_PRIORITY_CONFIG | RW | 32 | 0X0 | Configures channel priority.<br>Bits [2:0]: read top-priority channel;<br>bit [3]: read top-priority enable;<br>bits [6:4]: read high-priority channel;<br>bit [7]: read high-priority enable;<br>bits [10:8]: write top-priority channel;<br>bit [11]: write top-priority enable;<br>bits [14:12]: write high-priority channel;<br>bit [15]: write high-priority enable.<br>Bits [31:16] read as zero. |
| 0x103C | Reserved | — | — | — | Unmapped. APB access returns a slave error. |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | | | | |
|---|---|---|---|---|---|
| 0x1040 | CORE_CLOCK_DIVIDER | RW | 32 | 0X0 | Compatibility register. Clock-divider operation is not implemented in the current RTL. Reads return zero and writes have no functional effect. |
| 0x1044 | Reserved | — | — | — | Unmapped. APB access returns a slave error. |
| 0x1048 | CORE_CHANNEL_START | WO | 32 | 0X0 | Starts one or more DMA channels. Writing **1** to bit **n** generates a start pulse for Channel **n**. Bits [7:0] correspond to Channels 7 through 0. Bits [31:8] shall be written as zero. |
| 0x104C | Reserved | — | — | — | Unmapped. APB access returns a slave error. |
| 0x1050 | PERIPHERAL_RX_REQUEST | RW | 32 | 0X0 | Software-generated receive requests. Bits [31:1] correspond to RX peripheral IDs 31 through 1. Bit [0] is reserved because peripheral ID 0 represents memory. A request bit is cleared when the corresponding RX clear event occurs. |
| 0x1054 | PERIPHERAL_TX_REQUEST | RW | 32 | 0X0 | Software-generated transmit requests. Bits [31:1] correspond to TX peripheral IDs 31 through 1. Bit [0] is reserved. A request bit is cleared when the corresponding TX clear event occurs. |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | | | | |
|---|---|---|---|---|---|
| 0x1058–0x10CC | Reserved | — | — | — | Unmapped address range. APB access returns a slave error. |
| 0x10D0 | CORE_IDLE_STATUS | RO | 32 | 0x00000001 | Bit [0] reports the core idle state: 1 indicates that the DMA core is idle; 0 indicates that at least one DMA operation is active. Bits [31:1] read as zero. |
| 0x10D4–0x10DC | Reserved | — | — | — | Unmapped. APB access returns a slave error. |
| 0x10E0 | DESIGN_CAPABILITY_STATUS | RO | 32 | Fixed | Reports top-level implementation capabilities. Bits [3:0] report the number of interrupt destinations and are fixed to 1. Other implemented capability bits are fixed according to the current single-core wrapper. |
| 0x10E4–0x10EC | Reserved | — | — | — | Unmapped. APB access returns a slave error. |
| 0x10F0 | CORE_CAPABILITY_STATUS0 | RO | 32 | Fixed | Reports the fixed core configuration. Bits [3:0]: channel count=8;<br>[7:4]: FIFO size log2=5, corresponding to 32 bytes;<br>[11:8]: write-command depth log2=2;<br>[15:12]: read-command depth = 2<br>[21:16]: address width=32; |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | | | | |
|---|---|---|---|---|---|
| | | | | | bit [22]: 32-bit AXI data bus supported;<br>[28:24]: transfer-size field width=10.<br>Total = 0x0A60_2258 |
| 0x10F4 | CORE_CAPABILITY_STATUS1 | RO | 32 | Fixed | Reports optional features.<br>Bit [0]: watchdog;<br>[1]: AXI timeout;<br>[2]: token control;<br>[3]: priority control;<br>[4]: outstanding control =1 ;<br>[5]: wait control;<br>[8]: independent mode;<br>[9]: peripheral control;<br>[10]: command-list support;<br>[11]: endian swap;<br>[12]: clock divider. |
| 0x10F8–0x10FC | Reserved | — | — | — | Unmapped. APB access returns a slave error. |

**Table 7-2. DMA Channel Address Map**

| Address range | Register region |
|---|---|
| 0x0000–0x00FF | Channel 0 |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | |
|---|---|
| 0x0100–0x01FF | Channel 1 |
| 0x0200–0x02FF | Channel 2 |
| 0x0300–0x03FF | Channel 3 |
| 0x0400–0x04FF | Channel 4 |
| 0x0500–0x05FF | Channel 5 |
| 0x0600–0x06FF | Channel 6 |
| 0x0700–0x07FF | Channel 7 |
| 0x0800–0x0FFF | Unimplemented; APB access returns a slave error |
| 0x1000–0x10FF | Global register bank |

**Table 7-3. DMA Channel Register Map**

| Channel offset | Register | Type | Width | Reset | Description |
|---|---|---|---|---|---|
| 0x00 | CH_CMD_READ_ADDR | RW | 32 | 0X0 | Source address of the current DMA command. For memory accesses, this is the source-buffer address. For peripheral-to-memory operation, this is normally the fixed peripheral FIFO or data-register address. |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | | | | |
|---|---|---|---|---|---|
| 0x04 | CH_CMD_WRITE_ADDR | RW | 32 | 0X0 | Destination address of the current DMA command. For memory-to-peripheral operation, this is normally the fixed peripheral FIFO or data-register address. |
| 0x08 | CH_CMD_TRANSFER_SIZE | RW | 32 | 0X0 | Transfer amount in bytes.<br>Bits [9:0] contain the programmed transfer size.<br>Bits [31:10] read as zero. |
| 0x0C | CH_CMD_CONTROL | RW | 32 | 0X0 | Command control.<br>Bit [0], CMD_SET_INT: generate a completion interrupt when the command completes.<br>Bit [1], CMD_LAST: stop after this command when set; otherwise load the next command.<br>Bits [31:2] contain the next-command address. The generated command address is aligned to 4 bytes. |
| 0x10 | CH_READ_CONFIG | RW | 32 | 0x8401 0000 | Read-side configuration.<br>Bits [6:0]: maximum read burst size in bytes; The issued burst can be reduced by alignment, remaining transfer bytes, AXI 16-beat limit, 4 |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | | | | |
|---|---|---|---|---|---|
| | | | | | KB boundary, FIFO reservation, and peripheral service credit.<br>bits [21:16]: read token count;<br>bits [27:24]: configured read outstanding limit;<br>bit [30]: read outstanding enable;<br>bit [31]: source-address increment enable.<br>Other bits read as zero.<br>Total = 0x0000_0F1F |
| 0x14 | CH_WRITE_CONFIG | RW | 32 | 0x8401 0000 | Write-side configuration.<br>Bits [6:0]: Maximum read burst size in bytes.<br>bits [21:16]: write token count;<br>bits [27:24]: configured write outstanding limit;<br>bit [30]: write outstanding enable;<br>bit [31]: destination-address increment enable.<br>Other bits read as zero. |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | | | | |
|---|---|---|---|---|---|
| 0x18 | CH_MODE_CONFIG | RW | 32 | 0X0 | Channel mode configuration.<br>Bits [29:28], END_SWAP, select endian conversion: 00 no swap, 01 swap within 16 bits, 10 swap within 32 bits, 11 reserved for the current 32-bit datapath.<br>Other legacy mode fields have no functional effect and read as zero. |
| 0x1C | CH_SCHEDULE_CONFIG | RW | 32 | 0X0 | Compatibility register.<br>Programmable channel wait and scheduling functions are not implemented in the current RTL.<br>Reads return zero and writes have no functional effect. |
| 0x20 | CH_PERIPHERAL_CONFIG | RW | 32 | 0X0 | Peripheral selection and recovery delay.<br>Bits [4:0]: RD_PERIPH_NUM;<br>bits [10:8]: RD_PERIPH_DELAY;<br>bits [20:16]: WR_PERIPH_NUM;<br>bits [26:24]: WR_PERIPH_DELAY.<br>Peripheral ID 0 selects memory. |
| 0x24 | CH_AXI_ATTR_REG | RW | — | 0x0 | AXI attribute config<br>[3:0]: RD_CACHE<br>[5:4] RD_PROT<br>[7] RD_OVERRIDE_EN |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | | | | |
|---|---|---|---|---|---|
| | | | | | [11:8] WR_CACHE<br>[13:12] WR_PROT<br>[15] WR_OVERRIDE_END<br>[31:16]: Reserved |
| 0x2C | CH_RESTRICTION_STATUS | RO | 32 | 0X0 | Reports effective channel restrictions.<br>Bit [0]: source address permits full-FIFO burst;<br>bit [1]: destination address permits full-FIFO burst;<br>bit [2]: both addresses permit full-FIFO burst;<br>Bit [5]: effective read multi-pending mode active. Set only when the read endpoint is memory, read pending is enabled, and the effective read limit is greater than one.<br>Bit [6]: effective write multi-pending mode active. Set only when the write endpoint is memory, write pending is enabled, and the effective write limit is greater than one.<br>Bit [8]: memory-to-memory transfer configuration active. This bit does not gate bits [5] or [6].<br>Other bits read as zero. |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | | | | |
|---|---|---|---|---|---|
| 0x30 | CH_READ_OFFSET_STATUS | RO | 32 | 0X0 | Reserved status location. The current RTL always returns zero. |
| 0x34 | CH_WRITE_OFFSET_STATUS | RO | 32 | 0X0 | Reserved status location. The current RTL always returns zero. |
| 0x38 | CH_FIFO_STATUS | RO | 32 | Implementation-defined | FIFO status.<br>Bits [5:0]: read-side reservation capacity in bytes. This value excludes space already reserved for accepted or queued read work.<br>Bits [21:16]: write-side unassigned data capacity in bytes. This value reports received FIFO data that has not yet been allocated to a write command. |
| 0x3C | CH_OUTSTANDING_STATUS | RO | 32 | 0X0 | Accepted AXI transaction status.<br>[3:0] Number of AR transactions accepted for this channel whose final read response has not completed.<br>[11:8] Number of AW transactions accepted for this channel whose B<br>response has not completed.<br>Other bits read as zero. |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | | | | |
|---|---|---|---|---|---|
| 0x40 | CH_ENABLE | RW | 32 | 0x0000 0001 | Channel enable control.<br>Bit [0]: 1 enables channel operation; 0 prevents the channel from receiving new arbitration grants.<br>Bits [31:1] read as zero. |
| 0x44 | CH_START | WO | 32 | 0X0 | Channel software-start command.<br>Writing 1 to bit [0] generates a start event for the channel. |
| 0x48 | CH_ACTIVE _STATUS | RO | 32 | 0X0 | Channel active status.<br>Bit [0]: read side active.<br>Bit [1]: write side active. Bits [31:2] read as zero. |
| 0x4C | Reserved | — | — | — | Unmapped. APB access returns a slave error. |
| 0x50 | CH_TRANSF ER_COUNT | RO | 32 | 0X0 | Transfer counters.<br>Bits [11:0] report the number of completed commands since the most recent channel start.<br>Bits [19:16] report the number of pending completion-interrupt events.<br>Other bits read as zero. |
| 0x54–0x9C | Reserved | — | — | — | Unmapped address range. APB access returns a slave error. |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | | | | |
|---|---|---|---|---|---|
| 0xA0 | CH_INTERRUPT_RAW_STATUS | RW | 32 | 0X0 | Raw interrupt status. Bits [12:0] contain the interrupt sources. Software may write this register to set raw-status bits for debug or verification. |
| 0xA4 | CH_INTERRUPT_CLEAR | WO | 32 | 0X0 | Interrupt clear register. Writing 1 to a bit clears the corresponding raw interrupt source. |
| 0xA8 | CH_INTERRUPT_ENABLE | RW | 32 | 0x0000 1FFF | Interrupt enable mask.<br>Bits [12:0] enable the corresponding interrupt sources.<br>Bits [31:13] read as zero. |
| 0xAC | CH_INTERRUPT_STATUS | RO | 32 | 0X0 | Masked interrupt status calculated as CH_INTERRUPT_RAW_STATUS AND CH_INTERRUPT_ENABLE. Bits [12:0] correspond to the implemented interrupt sources. |
| 0xB0–0xFC | Reserved | — | — | — | Unmapped address range. APB access returns a slave error. |

## Interrupt bit assignment

The following interrupt assignment applies to CH_INTERRUPT_RAW_STATUS, CH_INTERRUPT_CLEAR, CH_INTERRUPT_ENABLE, and CH_INTERRUPT_STATUS:

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| Bit | Interrupt source |
|---|---|
| 0 | Channel command completed |
| 1 | AXI read SLVERR |
| 2 | AXI write SLVERR |
| 3 | AXI read DECERR |
| 4 | AXI write DECERR |
| 5 | Channel FIFO overflow |
| 6 | Channel FIFO underflow |
| 7 | AXI read-data timeout |
| 8 | AXI read-address timeout |
| 9 | AXI write-response timeout |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | |
|---|---|
| 10 | AXI write-data timeout |
| 11 | AXI write-address timeout |
| 12 | Channel watchdog timeout |
| 31:13 | Reserved |

# 8. Operation

The ftel_dma controller is programmed through the APB interface. DMA payload transfers are performed through the AXI master interface after software has configured and started one or more channels.

For Channel n, where n = 0...7, the register address is:

Channel Register Address =

DMA_BASE + (0x100 × n) + Channel_Register_Offset

Global registers are located at:

Global Register Address =

DMA_BASE + 0x1000 + Global_Register_Offset

Software shall not modify the command or static-configuration registers of a channel while that channel has pending AXI transactions

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

### 8.1. Initial Sequence

The initialization sequence prepares the DMA controller and all software-owned channels before the first transfer.

Although CH_ENABLE resets to 1 in the current RTL, software should explicitly disable the channel before programming it. This prevents a previous or unintended channel context from being started while its registers are being updated.

Table 9-1. DMA Initialization Sequence

| Step | Software action | Expected controller behavior |
|---|---|---|
| 1 | Assert the DMA reset during SoC initialization. Keep APB and AXI transactions inactive while reset is asserted. | Channel state, transfer counters, interrupt status, FIFO state, software peripheral requests, and internal control state are reset. |
| 2 | Deassert reset after the APB, DMA, and AXI clocks are stable. | The APB register interface becomes accessible. No AXI transfer is generated until a channel is started. |
| 3 | Read **DESIGN_CAPABILITY_STATUS** at **DMA_BASE** + 0x10E0, **CORE_CAPABILITY_STATUS0** at **DMA_BASE** + 0x10F0, and **CORE_CAPABILITY_STATUS1** at DMA_BASE + 0x10F4. | Software obtains the implemented channel count, FIFO size, address width, AXI data width, transfer-size width, and optional feature information. |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | |
|---|---|---|
| 4 | Read **CORE_IDLE_STATUS** at DMA_BASE + 0x10D0. | Bit [0] = 1 indicates that the DMA core is idle. |
| 5 | For each software-owned Channel ***n***, write 0X0 to **CH_ENABLE** at **DMA_BASE** + 0x100×n + 0x40. | The selected channel is prevented from issuing new DMA operations while it is configured. |
| 6 | For a previously used channel, poll **CH_OUTSTANDING_STATUS** at offset 0x3C until both read and write outstanding counts are zero. | Poll both per-channel pending counts to zero. Disabling a channel prevents<br>new grants but does not cancel AXI transactions already accepted. |
| 7 | Write 0x00001FFF to **CH_INTERRUPT_CLEAR** at offset 0xA4. | All stale completion, AXI error, FIFO error, timeout, and watchdog interrupt sources are cleared. |
| 8 | Program **CH_INTERRUPT_ENABLE** at offset 0xA8. Write 0x00000001 for completion-only operation, or 0x00001FFF to enable all currently implemented interrupt sources. | Only enabled raw interrupt sources contribute to CH_INTERRUPT_STATUS and the consolidated DMA interrupt output. |
| 9 | Program the command registers: CH_CMD_READ_ADDR, CH_CMD_WRITE_ADDR, CH_CMD_TRANSFER_SIZE, and CH_CMD_CONTROL. | Software shall not modify command, endpoint, burst, pending-limit, or<br>attribute configuration while either CH_ACTIVE_STATUS is nonzero or CH_OUTSTANDING_STATUS is nonzero. |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | |
|---|---|---|
| 10 | Program CH_READ_CONFIG, CH_WRITE_CONFIG, CH_MODE_CONFIG, CH_SCHEDULE_CONFIG, and CH_PERIPHERAL_CONFIG as required by the transfer direction. | The controller stores the source/destination addressing modes, AXI burst limits, token limits, outstanding limits, peripheral selections, and peripheral delays. |
| 11 | Optionally read back the programmable registers and verify the values. | Software confirms that the APB writes were accepted before enabling the channel. |
| 12 | Write 0x00000001 to CH_ENABLE. | The channel becomes eligible for operation but does not start until a start command is received. |
| 13 | Start the channel by writing 0x00000001 to **CH_START** at offset 0x44, or write bit ***n*** to **CORE_CHANNEL_START** at DMA_BASE + 0x1048. | The channel initializes its transfer-progress state, clears its command/completion counters, and becomes active. |
| 14 | Read **CH_ACTIV**<br>**E_STATUS** at offset 0x48 if start confirmation is required. | Bit [0] indicates read-side activity and bit [1] indicates write-side activity. |

### 8.2. Software Sequence

**Common channel programming order**

For Channel *n*, software should use the following order:

1. CH_ENABLE = 0

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

2. Wait outstanding count = 0

3. CH_INTERRUPT_CLEAR = 0x00001FFF

4. CH_INTERRUPT_ENABLE = required interrupt mask

5. CH_CMD_READ_ADDR = source address

6. CH_CMD_WRITE_ADDR = destination address

7. CH_CMD_TRANSFER_SIZE = transfer size in bytes

8. CH_CMD_CONTROL = command control

9. CH_READ_CONFIG = source configuration

10. CH_WRITE_CONFIG = destination configuration

11. CH_MODE_CONFIG = mode configuration

12. CH_SCHEDULE_CONFIG = scheduling configuration

13. CH_PERIPHERAL_CONFIG = peripheral selection

14. CH_ENABLE = 1

15. CH_START = 1

CH_START may be replaced by setting the channel bit in the global CORE_CHANNEL_START register.

### 8.3.1 Common Register Field Configuration

#### CH_READ_CONFIG and CH_WRITE_CONFIG

The active fields implemented by the current RTL are:

| Field | Bits | Description |
|---|---|---|

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | |
|---|---|---|
| BURST_MAX_SIZE | [6:0] | Maximum burst size in bytes. |
| TOKENS | [21:16] | Maximum number of consecutive AXI commands before arbitration release. |
| OUTS_MAX | [27:24] | Configured outstanding limit. Hardware clamps the effective value to 1–4. |
| OUTSTANDING_ENABLE | [30] | RD_OUTSTANDING_ENABLE applies to the read endpoint independently. If the source endpoint is memory, the read pending limit may be 1–4. If the source endpoint is a peripheral, the effective limit is 1. WR_OUTSTANDING_ENABLE applies to the write endpoint independently. If the destination endpoint is memory, the write pending limit may be 1–4. If the destination endpoint is a peripheral, the effective limit is 1. |
| ADDRESS_INCREMENT | [31] | 1: increment address; 0: fixed address. |

Example for incrementing address and an 8-byte maximum burst:

CH_x_CONFIG = 0x81010008

Example for fixed address and a 4-byte maximum burst:

CH_x_CONFIG = 0x01010004

Reserved bits should be written as zero.

**CH_PERIPHERAL_CONFIG**

Bits [4:0] RD_PERIPH_NUM

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

Bits [10:8] RD_PERIPH_DELAY

Bits [20:16] WR_PERIPH_NUM

Bits [26:24] WR_PERIPH_DELAY

Peripheral number 0 selects memory.

### 8.3.2 Memory-to-Memory Sequence

For memory-to-memory transfer:

RD_PERIPH_NUM = 0

WR_PERIPH_NUM = 0

Source address = Incrementing

Destination address = Incrementing

**Table 8-4. Memory-to-Memory Transfer Sequence**

| Step | Software action | Expected controller behavior |
|---|---|---|
| 1 | Disable the selected channel and clear stale interrupts. | The channel is ready for reconfiguration. |
| 2 | Write the source memory address to CH_CMD_READ_ADDR. | The AXI read path uses this address as the transfer source. |
| 3 | Write the destination memory address to CH_CMD_WRITE_ADDR. | The AXI write path uses this address as the transfer destination. |
| 4 | Write the transfer size in bytes to CH_CMD_TRANSFER_SIZE. | Read-side and write-side remaining-byte counters are initialized from this value when started. |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | |
|---|---|---|
| 5 | Write 0x00000003 to CH_CMD_CONTROL. | The channel stops after the current command and generates a completion event. |
| 6 | Configure both source and destination as incrementing addresses. Set peripheral numbers to zero. | No external peripheral request is required. |
| 7 | Enable and start the channel. | The read engine fills the channel FIFO and the write engine drains it to the destination. |
| | | |
| 8 | Wait for an interrupt or poll CH_ACTIVE_STATUS. | Completion occurs after all read data, write data, and write responses have completed. |
| 9 | Clear the completion interrupt. | The channel is ready for another command. |

**Example: Channel 0, 256-byte memory copy**

Source address = 0x00001000

Destination address = 0x00005000

Transfer size = 256 bytes

Burst maximum = 8 bytes

Completion IRQ = enabled

Register writes:

DMA_BASE + 0x0040 = 0X0 // CH_ENABLE = 0

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

DMA_BASE + 0x00A4 = 0x00001FFF  // Clear stale interrupts

DMA_BASE + 0x00A8 = 0x00001FFF  // Enable completion and errors

DMA_BASE + 0x0000 = 0x00001000  // CH_CMD_READ_ADDR

DMA_BASE + 0x0004 = 0x00005000  // CH_CMD_WRITE_ADDR

DMA_BASE + 0x0008 = 0x00000100  // 256 bytes

DMA_BASE + 0x000C = 0x00000003  // SET_INT=1, LAST=1

DMA_BASE + 0x0010 = 0x81010008  // Read: increment, normal, burst=8

DMA_BASE + 0x0014 = 0x81010008  // Write: increment, normal, burst=8

DMA_BASE + 0x0018 = 0X0  // Normal mode

DMA_BASE + 0x0020 = 0X0  // Memory to memory

DMA_BASE + 0x0040 = 0x00000001  // CH_ENABLE = 1

DMA_BASE + 0x0044 = 0x00000001  // CH_START = 1

### 8.3.3 Memory-to-Peripheral Sequence

A Memory-to-Peripheral transfer moves data from an incrementing memory buffer to a fixed memory-mapped peripheral data register or FIFO.

The transfer shall be configured as follows:

RD_PERIPH_NUM = 0

WR_PERIPH_NUM = 1...4

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

Source address = Incrementing

Destination address = Fixed

Peripheral numbers 1 through 4 correspond to the peripheral adapters instantiated in the current top-level wrapper.

**Table 8-5. Memory-to-Peripheral Transfer Sequence**

| Step | Software action | Expected controller behavior |
|---|---|---|
| 1 | Select an unused DMA channel and write 0 to CH_ENABLE. | The channel is prevented from starting while its configuration is updated. |
| 2 | Poll CH_OUTSTANDING_STATUS until the read and write outstanding counts are zero. | All previously accepted AXI transactions for the channel have completed. |
| 3 | Write 0x00001FFF to CH_INTERRUPT_CLEAR. | Previous completion and error interrupt sources are cleared. |
| 4 | Program CH_INTERRUPT_ENABLE with the required completion and error mask. | Enabled interrupt sources can propagate to the consolidated DMA interrupt output. |
| 5 | Write the source memory address to CH_CMD_READ_ADDR. | The AXI read engine uses this address as the source buffer. |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | |
|---|---|---|
| 6 | Write the peripheral FIFO or data-register address to CH_CMD_WRITE_ADDR. | The AXI write engine repeatedly accesses the selected peripheral address. |
| 7 | Write the number of bytes to transfer to CH_CMD_TRANSFER_SIZE. | The channel initializes its read-side and write-side remaining-transfer counters from this value. |
| 8 | Write 0x00000003 to CH_CMD_CONTROL. | The current command is treated as the last command and generates a completion interrupt when finished. |
| 9 | Program CH_READ_CONFIG with source-address increment enabled. | Memory read addresses advance after each completed read transfer. |
| 10 | Program CH_WRITE_CONFIG with destination-address increment disabled. | All AXI writes target the same peripheral FIFO or data-register address. |
| 11 | Write the selected peripheral number to CH_PERIPHERAL_CONFIG.WR_PERIPH_NUM. Set RD_PERIPH_NUM to zero. | Memory reads are always eligible. Peripheral writes are serviced through the selected TX adapter. |
| 12 | Program WR_PERIPH_DELAY if additional recovery cycles are required between two peripheral services. | The channel delays its next write-side request by the programmed number of DMA clock cycles. |
| 13 | Write 1 to CH_ENABLE, followed by 1 to CH_START. | The channel becomes active and begins reading source data into its dedicated FIFO. |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | |
|---|---|---|
| 14 | The peripheral submits a TX DMA request through its top-level request interface. | The corresponding TX adapter converts the peripheral request into the internal DMA request used by the selected channel. |
| 15 | The DMA writes data from the channel FIFO to the fixed peripheral address. | AXI write address, write data, and write response transactions are generated according to the channel configuration. |
| 16 | After the peripheral service operation completes, the adapter returns an acknowledgement to the requesting peripheral. | The peripheral may submit its next DMA request after completing the acknowledgement transaction. |
| 17 | Steps 14–16 repeat until the programmed transfer size reaches zero. | The DMA continues servicing peripheral requests while data remains in the command. |
| 18 | Wait for the completion interrupt or poll CH_ACTIVE_STATUS. | Completion is reported after the entire source buffer has been read and all destination writes have completed. |
| 19 | Clear the completion interrupt through CH_INTERRUPT_CLEAR. | The channel interrupt is removed and the channel can be programmed for another transfer. |

**Memory-to-Peripheral configuration**

The source and destination configurations should normally use:

CH_READ_CONFIG:

RD_INCR = 1

RD_OUTSTANDING_ENABLE = application choice

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

RD_OUTS_MAX = 1..4

RD_BURST_SIZE = memory prefetch burst size

CH_WRITE_CONFIG:

WR_INCR = 0

WR_OUTSTANDING_ENABLE = don't-care/0 recommended

effective WR pending limit = 1

WR_BURST_SIZE = bytes accepted for one peripheral service

The memory-read side may prefetch multiple bursts into the channel FIFO. The total reserved and buffered data cannot exceed 32 bytes. The peripheral-write side remains single-credit. The adapter does not accept another request until the current service is completed and its acknowledgement is accepted.

**WR_BURST_SIZE** should match the amount of data that the peripheral can accept for one DMA service. Typical values are:

| Peripheral data width | Recommended write burst size |
|---|---|
| 8-bit peripheral register | 1 byte |
| 16-bit peripheral register | 2 bytes |
| 32-bit peripheral register | 4 bytes |
| Peripheral FIFO | Number of bytes accepted per request, subject to RTL burst limits |

**Example: Channel 0, Memory-to-SPI0 TX**

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

Assume:

Source memory = 0x00001000

SPI0 TX data address = 0x40000000

Transfer size = 256 bytes

Peripheral number = 1

Peripheral unit = 1 byte

Example register sequence:

DMA_BASE + 0x0040 = 0X0 // CH_ENABLE = 0

DMA_BASE + 0x00A4 = 0x00001FFF // Clear previous interrupts

DMA_BASE + 0x00A8 = 0x00001FFF // Enable completion and error interrupts

DMA_BASE + 0x0000 = 0x00001000 // Source memory address

DMA_BASE + 0x0004 = 0x40000000 // SPI0 TX register/FIFO address

DMA_BASE + 0x0008 = 0x00000100 // Transfer 256 bytes

DMA_BASE + 0x000C = 0x00000003 // CMD_SET_INT=1, CMD_LAST=1

DMA_BASE + 0x0010 = 0x81010008 // Read memory, increment, burst max = 8 bytes

DMA_BASE + 0x0014 = 0x01010001 // Write peripheral, fixed, 1 byte/request

DMA_BASE + 0x0020 = 0x00010000 // WR_PERIPH_NUM = 1, SPI0 TX

DMA_BASE + 0x0040 = 0x00000001 // Enable Channel 0

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

DMA_BASE + 0x0044 = 0x00000001  // Start Channel 0

For USART0 TX, USART1 TX, or QSPI TX, only the peripheral address, peripheral transfer size, and WR_PERIPH_NUM need to be changed.

### 8.3.4 Peripheral-to-Memory Sequence

A Peripheral-to-Memory transfer moves data from a fixed peripheral data register or FIFO to an incrementing memory buffer.

**Table 8-6. Peripheral-to-Memory Transfer Sequence**

| Step | Software action | Expected controller behavior |
|---|---|---|
| 1 | Select an unused DMA channel and write 0 to CH_ENABLE. | The channel is disabled while its context is updated. |
| 2 | Poll CH_OUTSTANDING_STATUS until both outstanding counts are zero. | No transaction from the previous command remains pending. |
| 3 | Clear previous interrupt sources through CH_INTERRUPT_CLEAR. | The channel begins the new command without stale interrupt state. |
| 4 | Program CH_INTERRUPT_ENABLE. | Completion and selected error events are allowed to generate an interrupt. |
| 5 | Write the peripheral FIFO or data-register address to CH_CMD_READ_ADDR. | The read engine repeatedly accesses the same peripheral address. |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | |
|---|---|---|
| 6 | Write the destination memory-buffer address to CH_CMD_WRITE_ADDR. | Received data is stored into this memory region. |
| 7 | Write the required byte count to CH_CMD_TRANSFER_SIZE. | The DMA tracks the total amount of data to receive. |
| 8 | Write 0x00000003 to CH_CMD_CONTROL. | The channel stops and generates a completion event after the current command. |
| 9 | Configure CH_READ_CONFIG with source increment disabled. | The peripheral source address remains fixed. |
| 10 | Configure CH_WRITE_CONFIG with destination increment enabled. | The memory destination address advances after each write. |
| 11 | Write the selected peripheral number to CH_PERIPHERAL_CONFIG.RD_PERIPH_NUM. Set WR_PERIPH_NUM to zero. | Peripheral reads are controlled by the selected RX adapter; memory writes do not require a peripheral request. |
| 12 | Program RD_PERIPH_DELAY if the peripheral requires recovery cycles between DMA services. | The next read-side service is delayed by the programmed number of cycles. |
| 13 | Enable and start the channel. | The channel becomes active and waits for the selected peripheral RX request. |
| 14 | The peripheral submits an RX DMA request through its request interface. | The RX adapter converts the request into the DMA internal request. |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | |
|---|---|---|
| 15 | The DMA reads data from the fixed peripheral address into the channel FIFO. | An AXI read transaction is issued using the programmed peripheral transfer size. |
| 16 | The DMA writes the received data from the FIFO to the destination memory buffer. | AXI memory writes use incrementing destination addresses. |
| 17 | After the peripheral read service completes, the adapter returns an acknowledgement to the peripheral. | The peripheral may submit the next RX request. |
| 18 | Steps 14–17 repeat until the programmed transfer size is complete. | The destination memory buffer is filled progressively. |
| 19 | Wait for the completion interrupt or poll CH_ACTIVE_STATUS. | Completion is reported after all received data has been committed to memory. |
| 20 | Clear the completion interrupt. | The channel returns to an idle software-visible state. |

**Example: Channel 1, USART0 RX-to-Memory**

For Channel 1:

Channel base = DMA_BASE + 0x0100

Assume:

USART0 RX data address = 0x40001000

Destination memory = 0x00005000

Transfer size = 128 bytes

Peripheral number = 2

Peripheral unit = 1 byte

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

**Example register sequence:**

DMA_BASE + 0x0140 = 0X0  // CH_ENABLE = 0

DMA_BASE + 0x01A4 = 0x00001FFF  // Clear previous interrupts

DMA_BASE + 0x01A8 = 0x00001FFF  // Enable completion and error interrupts

DMA_BASE + 0x0100 = 0x40001000  // USART0 RX register/FIFO address

DMA_BASE + 0x0104 = 0x00005000  // Destination memory address

DMA_BASE + 0x0108 = 0x00000080  // Transfer 128 bytes

DMA_BASE + 0x010C = 0x00000003  // CMD_SET_INT=1, CMD_LAST=1

DMA_BASE + 0x0110 = 0x01010001  // Read peripheral, fixed, 1 byte/request

DMA_BASE + 0x0114 = 0x81010008  // Write memory, increment, burst max = 8 bytes

DMA_BASE + 0x0118 = 0X0  // No endian swap

DMA_BASE + 0x011C = 0X0  // No additional scheduling delay

DMA_BASE + 0x0120 = 0x00000002  // RD_PERIPH_NUM = 2, USART0 RX

DMA_BASE + 0x0140 = 0x00000001  // Enable Channel 1

DMA_BASE + 0x0144 = 0x00000001  // Start Channel 1

### 8.3.5 Software-Generated Peripheral Request

The DMA core still contains software peripheral-request registers:

PERIPHERAL_RX_REQUEST = DMA_BASE + 0x1050

PERIPHERAL_TX_REQUEST = DMA_BASE + 0x1054

These registers are useful for RTL verification or for configurations that connect the DMA request bus directly.

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

However, in the current SoC top-level wrapper, normal SPI0, USART0, USART1, and QSPI operation should use their peripheral adapter interfaces. Software should not normally generate a software request for these integrated peripheral paths unless this behavior is explicitly required and verified at the wrapper level.

**Table 8-7. Software Request Sequence**

| Step | Software action | Expected controller behavior |
|---|---|---|
| 1 | Configure and start the channel normally. | The channel becomes active and waits for its selected peripheral request. |
| 2 | Write bit p to PERIPHERAL_RX_REQUEST or PERIPHERAL_TX_REQUEST. | The corresponding software request is combined with the external request path inside the DMA core. |
| 3 | The DMA services the selected peripheral request. | The configured AXI read or write operation is performed. |
| 4 | The DMA generates the corresponding clear event. | The software request bit is cleared by the DMA request-clear mechanism. |

Software-generated requests should not be used simultaneously with an active adapter request for the same peripheral number unless this combination has been explicitly verified.

### 8.3.6 Multiple-Channel Start

Software can configure several channels independently and start them using a single global write.

**Table 8-8. Multiple-Channel Start Sequence**

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| Step | Software action | Expected controller behavior |
|---|---|---|
| 1 | Disable and configure all required channels independently. | Each channel stores its own command, addressing, peripheral, and interrupt configuration. |
| 2 | Clear stale interrupts for every selected channel. | Previous interrupt state does not affect the new group of transfers. |
| 3 | Enable all selected channels. | The channels become eligible for operation. |
| 4 | Write a channel bitmap to CORE_CHANNEL_START at DMA_BASE + 0x1048. | Every channel whose corresponding bit is one receives a software start event. |
| 5 | Monitor CORE_STATUS and the per-channel status registers. | The independent read and write arbiters schedule the active channels according to readiness and FIFO state. |

**Example: start Channels 0, 2, and 5:**

DMA_BASE + 0x1048 = 0x00000025

The read and write arbiters independently select eligible channels. Therefore, one channel may use the read engine while another channel simultaneously uses the write engine.

### 8.3.7 Channel Completion Sequence

Table 9-10. Channel Completion Sequence

| Step | Software action | Expected controller behavior |
|---|---|---|

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | |
|---|---|---|
| 1 | Wait for the DMA interrupt or poll CH_ACTIVE_STATUS. | The channel remains active while read-side or write-side work is pending. |
| 2 | Read CH_INTERRUPT_STATUS. | Bit [0] indicates a completed command when the completion interrupt is enabled. |
| 3 | Read CH_TRANSFER_COUNT. | Software obtains the number of completed commands and unserviced completion events. |
| 4 | For a Peripheral-to-Memory transfer, process the completed destination buffer. | Data written by the DMA is available to software after the required system coherency operations. |
| 5 | For a Memory-to-Peripheral transfer, mark or release the transmitted source buffer. | The programmed source data has been accepted by the destination peripheral path. |
| 6 | Write bit [0] to CH_INTERRUPT_CLEAR. | One pending channel-completion event is cleared. |
| 7 | If additional completion events remain, continue servicing them. | The completion interrupt remains asserted or is reasserted until all pending events are cleared. |

**M2P completion**

All source reads complete, all FIFO data is transferred, and the final destination B response is accepted.

**P2M completion**

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

All peripheral read services complete and all destination memory B responses are accepted.

### 8.3.8 Error Recovery Sequence

Table 9-11. DMA Error Recovery Sequence

| Step | Software action | Expected controller behavior |
|---|---|---|
| 1 | Read CORE_STATUS to identify the channel reporting the interrupt. | The active channel interrupt bitmap is returned. |
| 2 | Read CH_INTERRUPT_STATUS and CH_INTERRUPT_RAW_STATUS. | Software identifies whether the cause is an AXI error, timeout, watchdog timeout, FIFO overflow, or FIFO underflow. |
| 3 | Write 0 to CH_ENABLE. | The channel is prevented from issuing new transfer work. |
| 4 | Poll CH_OUTSTANDING_STATUS until both counts are zero. | Internal protocol errors may be reported through the read/write DECERR<br>interrupt bits. The current register map does not separately identify RID/BID mismatch, malformed RLAST, W-data fetch-credit error, or WSTRB alignment error. |
| 5 | Record CH_ACTIVE_STATUS, CH_FIFO_STATUS, address registers, transfer offsets, and interrupt status. | Diagnostic state is captured before the channel is reprogrammed. |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

| | | |
|---|---|---|
| 6 | Clear the handled interrupt sources through CH_INTERRUPT_CLEAR. | The corresponding raw interrupt bits are removed. |
| 7 | Correct the invalid address, burst size, peripheral selection, timeout cause, or outstanding configuration. | The channel receives a valid configuration for the retry. |
| 8 | Reprogram the command registers if the entire transfer must be restarted. | Source address, destination address, and transfer amount return to the required restart point. |
| 9 | Write 1 to CH_ENABLE, followed by 1 to CH_START. | The channel begins the corrected command. |

For peripheral transfers, software should verify that the corresponding peripheral adapter is no longer holding a previous transaction before restarting the channel. A system-level peripheral reset or request cancellation may be required when the peripheral itself caused the timeout.

# 9. Verification Notes

**[Guideline]**

Describe the specific requirements or corner cases for the DV team.

# 10. Integration

## 10.1. Primitive Modules

**[Guideline]**

List up all primitive modules that must be replaced by the technology cells

## 10.2. SDC Constraint

**[Guideline]**

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

Describe the specific requirements for the synthesis constraint. For example, you have the two async clock domains, and this requires the set_max_delay between them. You must have the diagram and description for this.

**[Example]**

To ensure that the address remains stable before the o_araddr register captures it by the read request, a maximum delay of one receive clock cycle is required.

To ensure that the read data and response remain stable before the read data and response register captures it by the read acknowledge, a maximum delay of one receive clock cycle is required.

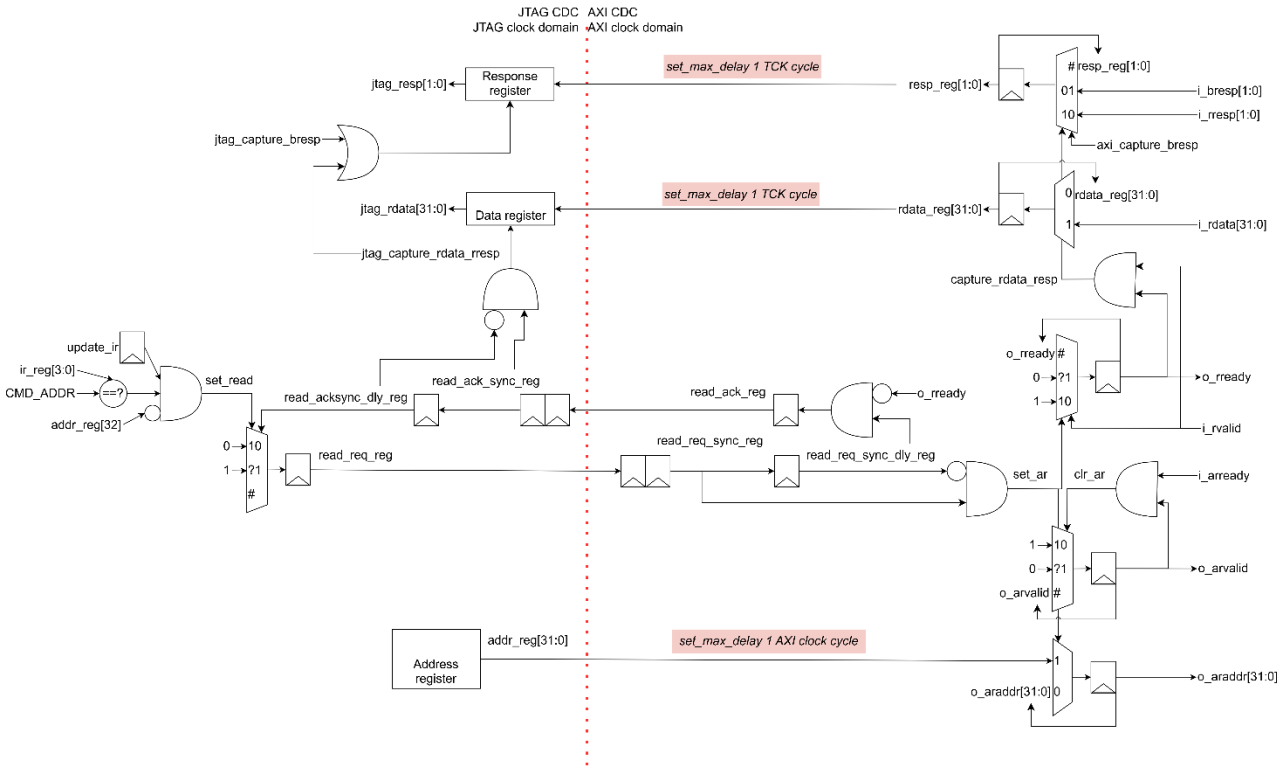



**Figure 11-1. Max Delay Constraint of JTAG-to-AXI Bridge**

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

# Appendix A. **Acronyms**

| Acronyms | Description |
|---|---|
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |

Copyright © 2026 FPT Telecom. All rights reserved. Confidential

# Appendix B. **Review**

**Table 11-1. Design Review 1**

| Version | Reviewer | Answer |
|---|---|---|
| V1.0 | Removal of certain features that are otherwise standard : Secure boot, OTP/eFUSE, debug lock, authentification, ... are standard features for a modern MCU. | Regarding Secure Boot and OTP/eFuse, we are currently limited by the Design Kit (DK). Due to the lack of OTP/eFuse hardware for programming the authentication public key, and given the tight 3-month schedule which precludes securing an internal flash vendor, we will implement a workaround using immutable ROM code.<br>Similarly, while debugger locking typically requires an OTP bit for production release, we will utilize System Configuration Registers (SYSREG) to disable the debugging interface as a functional alternative.<br>Conclusion: We don't have a perfect solution when the OTP is not ready. |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |

Startpoint:
u_fpt_dma0_wrap_0/u_dma_top_0/m_fpt_dma_core_wrap/m_fpt_dma_core/m_fpt_dma_core_engine/m_fpt_dma_channels/m_fpt_dma_channel1/m_fpt_dma_channel_reg/rd_periph_num_reg_reg_0_

(rising edge-triggered flip-flop clocked by clk_system)

Endpoint:
u_fpt_dma0_wrap_0/u_dma_top_0/m_fpt_dma_core_wrap/m_fpt_dma_core/m_fpt_dma_core_engine/m_fpt_dma_arbiter_rd/ch_last_reg

(rising edge-triggered flip-flop clocked by clk_system)

Path Group: clk_system

Path Type: max

Copyright © 2026 FPT Telecom. All rights reserved. Confidential